



SINCE 1947

서경대학교

SEOKYEONG UNIVERSITY

Term Project 최종 보고서

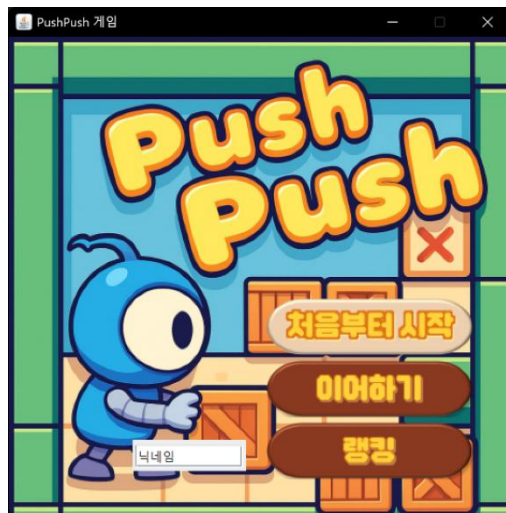
Push Push 게임 만들기

전공	소프트웨어학과	학번	2024216026
과목명	자바프로그래밍	담당 교수님	신우창
제출일	2025.06.17	이름	김찬희

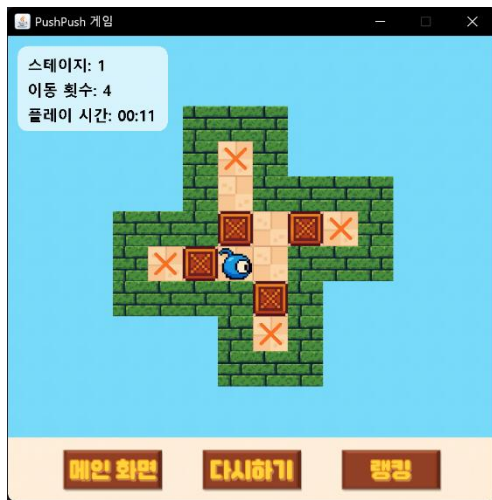
1. 사용 매뉴얼



- (1) 게임을 시작하면, "닉네임" 란에 이름을 입력하세요. 이름을 입력하지 않으면 "닉네임" 이름으로 플레이합니다.



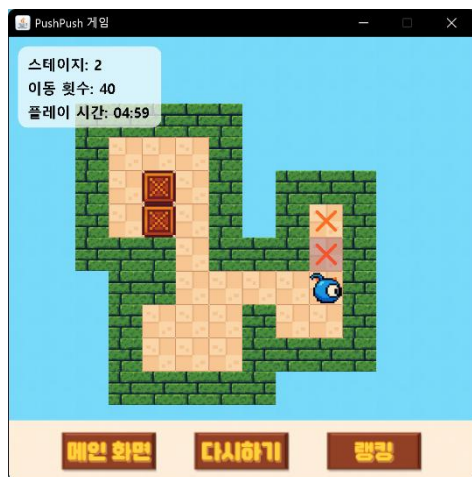
- (2) 누르고자 하는 버튼에 커서를 가져다 대면 효과음과 함께 반응합니다. 처음부터 시작을 눌러 플레이하세요.



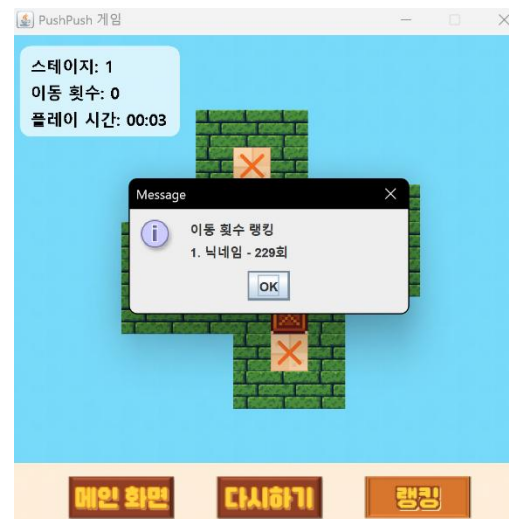
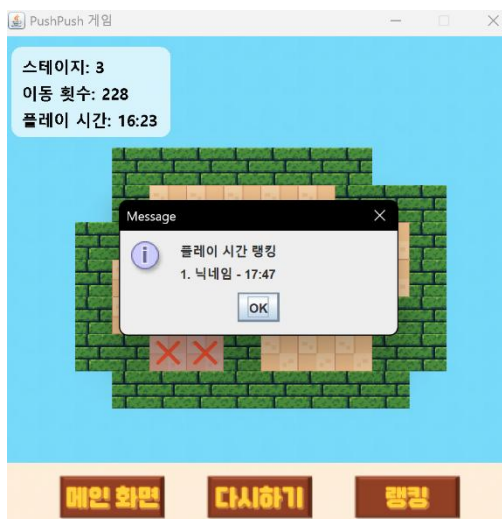
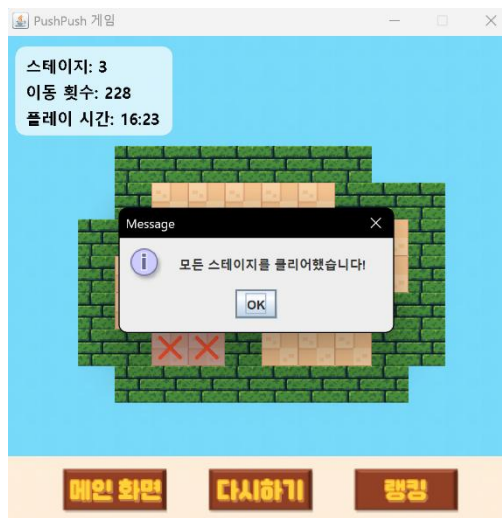
(3) 키보드의 $\leftarrow \uparrow \downarrow \rightarrow$ 키를 활용하여 움직이세요! 상자를 X표시된 곳에 모두 옮기면 클리어입니다.

(4) 왼쪽 상단에서 스테이지 정보와 이동 횟수, 플레이 시간을 확인하세요.

(5) 하단의 메인 화면 버튼을 클릭해 메인 화면으로 되돌아갈 수 있고, 다시하기 버튼을 통해 해당 스테이지를 다시 플레이할 수 있습니다.



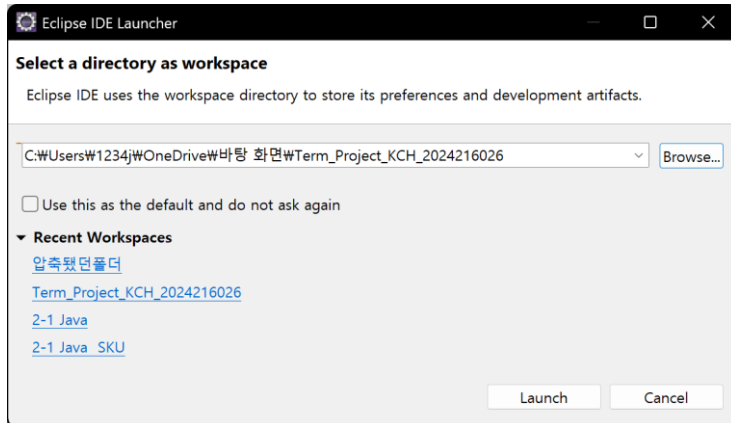
(6) 상자를 목표 지점까지 밀면, 목표 지점이 빨강게 변합니다. 상자를 밀어 목표 지점에 도착한 상자를 이동시키고 다음 상자를 차례로 집어넣을 수 있습니다.



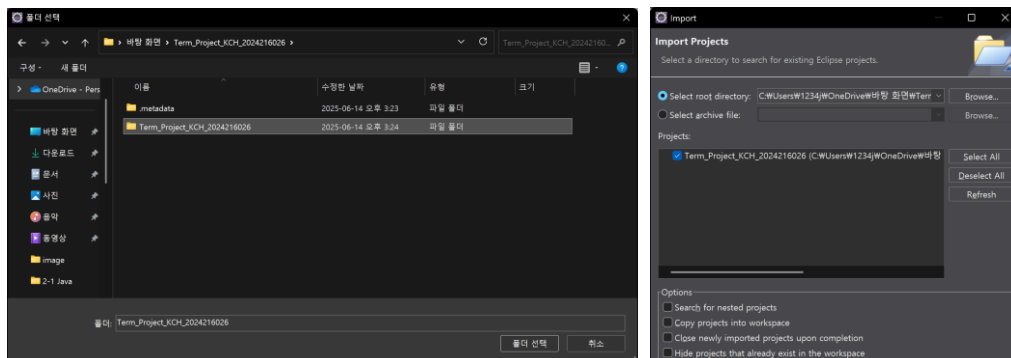
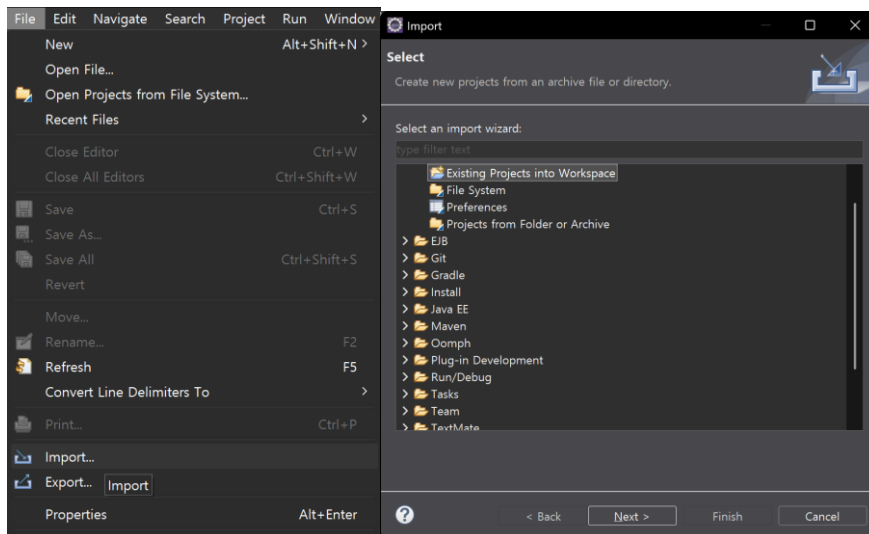
(7) 모든 스테이지를 클리어하면 클리어 팝업과 함께 랭킹에 "닉네임"이 기록됩니다. 랭킹은 플레이 시간 랭킹과 이동 횟수 랭킹으로 나누어져 있습니다.

2. 개발 매뉴얼

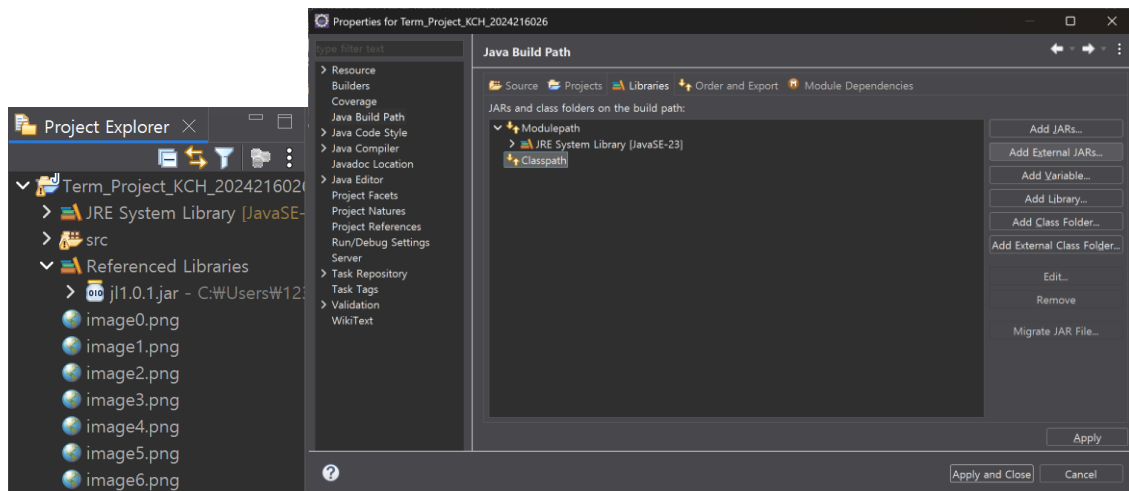
2-0 파일 세팅



(1) 파일 압축을 해제하고 이클립스에서 압축 해제된 폴더를 선택합니다.



- (2) 왼쪽 상단의 file에서 import를 클릭하고, Existing Projects into Workspace을 눌러 폴더 안의 Term_Project 폴더를 선택하고 Finish를 누르세요.



- (3) JLayer 외부 라이브러리를 이미 설치해 두었습니다. 설치된 경로가 달라 오류가 나면 jl1.0.1.jar파일을 build path에서 삭제한 후 제공된 jl1.0.1.jar 파일을 classpath에 Add External JARs를 통해 다시 추가하세요.
- (4) 프로젝트가 실행되지 않으면 JRE 버전을 확인하세요.

2-1. MainScreen.java

```
public class MainScreen extends JFrame {
    private CardLayout card;
    private JPanel ChangePanel;
    private GameMap map;

    private String lastPlayerName = "";
    private Image ScreenImage;
    private Graphics ScreenGraphic;

    private Image MainImage = new ImageIcon(getClass().getResource("../image/MainImage.png")).getImage(); //참조 찾아서 image받는 방법 - joyful velog 참고
    private Music MainMusic = new Music("MainMusic.mp3", true);

    private Image GameImage = new ImageIcon(getClass().getResource("../image/GameImage.png")).getImage();
    private Music GameMusic = new Music("GameMusic.mp3", true);

    private ImageIcon MainNewBtn0 = new ImageIcon(getClass().getResource("../image/MainNewBtn0.png"));
    private ImageIcon MainNewBtn1 = new ImageIcon(getClass().getResource("../image/MainNewBtn1.png"));
    private ImageIcon MainContinueBtn0 = new ImageIcon(getClass().getResource("../image/MainContinueBtn0.png"));
    private ImageIcon MainContinueBtn1 = new ImageIcon(getClass().getResource("../image/MainContinueBtn1.png"));
    private ImageIcon MainRankBtn0 = new ImageIcon(getClass().getResource("../image/MainRankBtn0.png"));
    private ImageIcon MainRankBtn1 = new ImageIcon(getClass().getResource("../image/MainRankBtn1.png"));

    private ImageIcon GameMainBtn0 = new ImageIcon(getClass().getResource("../image/GameMainBtn0.png"));
    private ImageIcon GameMainBtn1 = new ImageIcon(getClass().getResource("../image/GameMainBtn1.png"));
    private ImageIcon GameResetBtn0 = new ImageIcon(getClass().getResource("../image/GameResetBtn0.png"));
    private ImageIcon GameResetBtn1 = new ImageIcon(getClass().getResource("../image/GameResetBtn1.png"));
    private ImageIcon GameRankBtn0 = new ImageIcon(getClass().getResource("../image/GameRankBtn0.png"));
    private ImageIcon GameRankBtn1 = new ImageIcon(getClass().getResource("../image/GameRankBtn1.png"));

    private JButton MainNewBtn = new JButton(MainNewBtn0);
    private JButton MainContinueBtn = new JButton(MainContinueBtn0);
    private JButton MainRankBtn = new JButton(MainRankBtn0);

    private JButton GameMainBtn = new JButton(GameMainBtn0);
    private JButton GameResetBtn = new JButton(GameResetBtn0);
    private JButton GameRankBtn = new JButton(GameRankBtn0);
}
```

(1) 객체 미리 선언

mp3파일을 재생하는 Music과 Image, ImageIcon을 각각 먼저 선언하여 생성자 외에 함수들에도 사용할 수 있도록 했다.

객체 지향적으로 작성하기 위해 클래스들을 각각의 역할을 수행하도록 나누면서, 이 파일이 메인 클래스임을 명확히 하기 위해 파일명을 MainScreen으로 변경했다.

```
public MainScreen() {
    setTitle("PushPush 게임");
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    card = new CardLayout();
    ChangePanel = new JPanel(card);

    map = new GameMap();

    setSize(500, 500);
    setResizable(false);
    setLocationRelativeTo(null);
    setLayout(null);
}
```

(2) CardLayout()을 통한 Layout 번갈아 보여주기

기존에 메인 화면 없이 바로 실행되던 코드에서, 메인 화면과 게임 화면을 번갈아 보여줄 수 있도록 하기 위해 CardLayout()을 사용했다. 교재에서 자세하게 다루지 않았기 때문에 인터넷에서 CardLayout() 사용법을 참조했다.


```

MainNewBtn.setBounds(150, 230, 400, 100);
MainNewBtn.setBorderPainted(false); // 버튼 모양 삭제 - 배경화면만지 블로그 참고
MainNewBtn.setContentAreaFilled(false);
MainNewBtn.setFocusPainted(false);
MainNewBtn.addMouseListener(new MouseAdapter() {
    public void mouseEntered(MouseEvent e) {
        MainNewBtn.setIcon(MainNewBtn0);
        Music BtnEnterMusic = new Music("BtnEnterMusic.mp3", false);
        BtnEnterMusic.start();
    }

    public void mouseExited(MouseEvent e) {
        MainNewBtn.setIcon(MainNewBtn0);
    }

    public void mousePressed(MouseEvent e) {
        Music BtnPressMusic = new Music("BtnPressMusic.mp3", false);
        BtnPressMusic.start();
        String inputName = NameField.getText().trim();
        if (inputName.equals("")) {
            JOptionPane.showMessageDialog(null, "이름을 입력하세요.");
            return;
        }
        lastPlayerName = inputName;
        map.getStatus().setLastPlayerName(lastPlayerName);
        map.startFromFirstStage();
        showGamePanel();
    }
});
Mainpanel.add(MainNewBtn);

```

(3) 버튼 기본 모양 삭제

교재에 나온 내용에서는 이미지를 삽입하는 방법까지만 나와있고, 버튼의 기본 모양을 삭제하여 이미지가 버튼을 완전히 대체하는 부분을 알려주지 않아 개발자 블로그를 참고하여 이미지 자체가 버튼이 될 수 있도록 했다.

버튼에 커서를 대거나 떼면 버튼 이미지가 바뀌고, 버튼에 커서를 대거나 누르면 효과음이 실행되도록 했다. 후술할 Music 클래스에 파일 이름과 boolean값을 줘서 true이면 무한 재생, false이면 한 번만 재생하도록 한다.

```

ChangePanel.add(Mainpanel, "main");

JPanel Gamepanel = new JPanel(new BorderLayout());
JPanel GameBtnpanel = new JPanel() {
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.drawImage(GameImage, 0, 0, getWidth(), getHeight(), this);
    }
};
Gamepanel.setSize(new Dimension(500, 50));

map.addKeyListener(new KeyAdapter() {
    public void keyPressed(KeyEvent e) {
        int keycode = e.getKeyCode();

        switch (keycode) {
            case KeyEvent.VK_UP:
                map.movePlayer(-1, 0);
                break;
            case KeyEvent.VK_DOWN:
                map.movePlayer(1, 0);
                break;
            case KeyEvent.VK_LEFT:
                map.movePlayer(0, -1);
                break;
            case KeyEvent.VK_RIGHT:
                map.movePlayer(0, 1);
                break;
        }
    }
});
Gamepanel.add(map, BorderLayout.CENTER);

```

(4) CardLayout과 버튼

Mainpanel에 배경화면과 버튼, 텍스트필드를 모두 넣고 나서 CardLayout에 "main" 이름으로 추가하고, Gamepanel을 생성하여 요소들을 삽입했다.

GameBtnpanel에 묶여 Gamepanel에 추가되는 다시하기, 랭크 버튼들은 마지막에 map에 focus하도록 해야 버튼을 클릭한 후에도 조작이 가능하다.

```

Gamepanel.add(GameBtnpanel, BorderLayout.SOUTH);

ChangePanel.add(Gamepanel, "game");
setContentPane(ChangePanel);
setVisible(true);
}

```

(5) 실행 시 나타나는 첫 화면 = 메인 화면

Gamepanel에 요소를 모두 넣은 후 CardLayout에 "game" 이름으로 추가하고, ContentPane을 CardLayout인 ChangePanel로 설정한다.

```

public void resetPlayerName() {
    lastPlayerName = "";
}

public void showMainPanel() {
    card.show(ChangePanel, "main");

    GameMusic.close();
    MainMusic = new Music("MainMusic.mp3", true);
    MainMusic.start();
}

public void showGamePanel() {
    card.show(ChangePanel, "game");
    map.requestFocusInWindow();

    MainMusic.close();
    GameMusic = new Music("GameMusic.mp3", true);
    GameMusic.start();
}

public static void main(String[] args) {
    new MainScreen();
}

```

(6) MainScreen의 함수

resetPlayerName() 함수는 모든 스테이지를 클리어하고 나면 실행되어 lastPlayerName을 초기화

showMainPanel() 함수는 생성자 위에 선언된 객체를 통해 메인 화면으로 전환, 음악도 메인 화면의 음악으로 재생

showGamePanel() 함수는 생성자 위에 선언된 객체를 통해 게임 화면으로 전환, 음악도 게임 화면의 음악으로 재생

2-2. Music.java

```
package term_project_final;

import java.io.BufferedReader;

public class Music extends Thread{
    private Player player;
    private boolean isloop;
    private File file;
    private FileInputStream fis;
    private BufferedInputStream bis;

    public Music(String name, boolean isloop) {
        try {
            this.isloop = isloop;
            file = new File(getClass().getResource("../Music/"+name).toURI());
            fis = new FileInputStream(file);
            bis = new BufferedInputStream(fis);
            player = new Player(bis);
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }

    public int getTime() {
        if (player == null) {
            return 0;
        }
        return player.getPosition();
    }

    public void close() {
        isloop = false;
        player.close();
        this.interrupt(); //곡 종료
    }

    @Override
    public void run() {
        try {
            do {
                player.play();
                fis = new FileInputStream(file);
                bis = new BufferedInputStream(fis);
                player = new Player(bis);
            } while (isloop);
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```

(1) Mp3 실행 라이브러리 설치

Mp3 파일을 실행하기 위한 라이브러리 JLayer을 설치 후 빌드 패스 설정하여 해당 코드를 작성하였다.

2-3. MapData.java

```
public class MapData {
    public static final int ROW_COUNT = 14;
    public static final int COL_COUNT = 14;

    // 맵 데이터
    private int[][][] stageMap = ( // 초기 맵 데이터
        { { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
          { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
          { 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0 },
          { 0, 0, 0, 0, 0, 1, 3, 1, 0, 0, 0, 0, 0, 0 },
          { 0, 0, 0, 0, 0, 1, 2, 1, 1, 1, 1, 0, 0, 0 },
          { 0, 0, 0, 1, 1, 1, 4, 2, 4, 3, 1, 0, 0, 0 },
          { 0, 0, 0, 1, 3, 2, 4, 5, 1, 1, 1, 0, 0, 0 },
          { 0, 0, 0, 1, 1, 1, 1, 4, 1, 0, 0, 0, 0, 0 },
          { 0, 0, 0, 0, 0, 1, 3, 1, 0, 0, 0, 0, 0, 0 },
          { 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0 },
          { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
          { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
          { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
          { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 } },

        { { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
          { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
          { 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0 },
          { 0, 0, 1, 5, 2, 2, 1, 0, 0, 0, 0, 0, 0, 0 },
          { 0, 0, 1, 2, 4, 4, 1, 0, 1, 1, 1, 0, 0, 0 },
          { 0, 0, 1, 2, 4, 2, 1, 0, 1, 1, 1, 0, 0, 0 },
          { 0, 0, 1, 1, 2, 1, 1, 1, 1, 1, 1, 0, 0, 0 },
          { 0, 0, 0, 1, 1, 2, 2, 2, 2, 1, 0, 0, 0, 0 },
          { 0, 0, 0, 1, 2, 2, 2, 1, 2, 2, 1, 0, 0, 0 },
          { 0, 0, 0, 1, 2, 2, 2, 1, 1, 1, 1, 0, 0, 0 },
          { 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0 },
          { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
          { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
          { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 } },

        { { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
          { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
          { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
          { 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0 },
          { 0, 0, 0, 1, 2, 2, 2, 2, 2, 1, 1, 1, 0, 0 },
          { 0, 0, 1, 1, 4, 1, 1, 1, 2, 2, 2, 1, 0, 0 },
          { 0, 0, 1, 2, 5, 2, 4, 2, 2, 4, 2, 1, 0, 0 },
          { 0, 0, 1, 2, 3, 3, 1, 2, 4, 2, 1, 1, 0, 0 },
          { 0, 0, 1, 1, 3, 3, 1, 2, 2, 2, 1, 1, 0, 0 },
          { 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0 },
          { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
          { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
          { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 },
          { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 } }
    );
}
```

```

private Image[] images;

public MapData() {
    images = new Image[7];
    for (int i = 0; i < images.length; ++i) {
        images[i] = new ImageIcon("image" + i + ".png").getImage();
    }
}

public int[][] getStage(int stageNum) {
    int[][] map = new int[ROW_COUNT][COL_COUNT];
    for (int r = 0; r < ROW_COUNT; r++)
        for (int c = 0; c < COL_COUNT; c++)
            map[r][c] = stageMap[stageNum][r][c];
    return map;
}

public Image[] getImages() {
    return images;
}

public int getStageCount() {
    return stageMap.length;
}
}

```

(1) 맵데이터 배열과 이미지 로드, getStage()와 getImages(),
getStageCount() 함수

스테이지별 맵데이터를 stageMap에 3차원 배열로 나타내고, 생성자에서
이미지 배열에 이미지들을 로드, 해당 스테이지를 복사해 주는
getStage()함수와 이미지 배열을 리턴해주는 getImages() 함수, 현재
스테이지의 단계를 리턴해 주는 getStageCount() 함수를 작성했다.

2-4. PlayerController.java

```

public class PlayerController {
    private int playerRow, playerCol;

    public PlayerController(int[][] map) {
        findPlayer(map);
    }

    private void findPlayer(int[][] map) {
        for (int r = 0; r < map.length; r++)
            for (int c = 0; c < map[0].length; c++)
                if (map[r][c] == 5) {
                    playerRow = r;
                    playerCol = c;
                    return;
                }
    }

    public int getPlayerRow() {
        return playerRow;
    }

    public int getPlayerCol() {
        return playerCol;
    }

    public boolean movePlayer(int dr, int dc, int[][] map, int[][] stageMap) {
        int nr = playerRow + dr, nc = playerCol + dc;
        if (!inRange(nr, nc, map.length, map[0].length))
            return false;
        if (map[nr][nc] == 1)
            return false;

        if (map[nr][nc] == 4 || map[nr][nc] == 6) {
            int br = nr + dr, bc = nc + dc;
            if (!inRange(br, bc, map.length, map[0].length))
                return false;
            if (map[br][bc] != 2 && map[br][bc] != 3)
                return false;

            map[br][bc] = (map[br][bc] == 3) ? 6 : 4;
            map[nr][nc] = 2;
        }

        map[playerRow][playerCol] = (stageMap[playerRow][playerCol] == 3) ? 3 : 2;
        playerRow = nr;
        playerCol = nc;
        map[playerRow][playerCol] = 5;
        Music PlayerMove = new Music("PlayerMove.mp3", false);
        PlayerMove.start();
        return true;
    }
}

```

(1) 데이터 상의 플레이어 이동

플레이어의 위치를 맵데이터 배열 상으로 이동시키는 클래스를 분리했다. 다른 코드는 중간 보고서 때의 코드들과 비슷하지만, 플레이어가 데이터 상으로 이동했을 때 이동하는 효과음을 재생하도록 했다.

2-5. GameMap.java

```
public class GameMap extends JPanel {
    private MapData mapData;
    private PlayerController playerCtrl;
    private GameStatus status;
    private int[][] map;

    public GameMap() {
        setFocusable(true);
        mapData = new MapData();

        status = new GameStatus();
        status.startTimer(this);
        loadStage(status.getStageNum());
    }

    public GameStatus getStatus() {
        return status;
    }

    public void continuePlayTime() {
        status.startTimer(this);
    }

    public void stopPlayTime() {
        status.stopTimer();
    }

    public void startFromFirstStage() {
        status.setStageNum(0); // 스테이지 번호 0으로
        status.resetMoveCount(); // 이동 횟수 0으로
        status.resetTimer();
        loadStage(status.getStageNum()); // 맵 로딩
    }

    public void loadStage(int num) {
        map = mapData.getStage(num);
        playerCtrl = new PlayerController(map);
        repaint();
    }
}
```

(1) 함수와 status 객체 생성

MainScreen에서 객체를 생성해서 사용하는 시간 시작/멈춤 함수와 스테이지 처음부터 로드하는 함수 등이 있고, 이 함수들은 status 객체를 생성하여 GameStatus 클래스에서 정보들을 받아와서 실행한다.

```
public void movePlayer(int dr, int dc) {
    if (playerCtrl.movePlayer(dr, dc, map, mapData.getStage(status.getStageNum()))) {
        status.incrementMove();
        repaint();
        if (isWin()) {
            if (status.nextStage()) {
                Music stageClear = new Music("StageClear.mp3", false);
                stageClear.start();
                loadStage(status.getStageNum());
            } else {
                Music gameClear = new Music("GameClear.mp3", false);
                gameClear.start();
                JOptionPane.showMessageDialog(this, "모든 스테이지를 클리어했습니다!"); //팝업 메시지
                status.addRanking(status.getLastPlayerName());
                showRankingDialog();
                status.stopTimer();

                JFrame topFrame = (JFrame) SwingUtilities.getWindowAncestor(this); //stack overflow 질문 게시판 참고
                ((MainScreen) topFrame).showMainPanel();
                ((MainScreen) topFrame).resetPlayerName();
            }
        }
    }
}
```

(2) movePlayer 함수의 변경점

movePlayer 함수에서 모든 스테이지를 클리어했을 때 팝업창이 뜨지 않던 현상을 수정했고, SwingUtilities.getWindowAncestor로 상위 JFrame에 접근해서 MainScreen의 함수를 실행시켜 메인 화면으로 돌아가고 닉네임을 초기화하도록 했다. 스테이지를 클리어하거나 모든 단계를 클리어했을 때 효과음이 재생되도록 했다.

GameStatus의 객체 status의 nextStage 함수를 실행시킨 값이 참이면 다음 스테이지를 로드하는데 필요한 코드들을 작성하고, false를 반환받았다면 이미 status의 변수 stageNum이 MaxstageNum과 같아졌으므로 게임을 클리어하는 코드를 작성했다.

```
@Override
public void paintComponent(Graphics g) {
    super.paintComponent(g);
    Image[] imgs = mapData.getImages();
    for (int r = 0; r < MapData.ROW_COUNT; r++)
        for (int c = 0; c < MapData.COL_COUNT; c++)
            drawCell(g, r, c, imgs);

    g.setColor(new Color(255, 255, 255, 180));
    g.fillRoundRect(10, 10, 150, 85, 20, 20);
    g.setColor(Color.BLACK);
    g.setFont(new Font("맑은 고딕", Font.BOLD, 15));
    g.drawString("스테이지: " + (status.getStageNum() + 1), 20, 35);
    g.drawString("이동 횟수: " + status.getMoveCount(), 20, 60);
    g.drawString("플레이 시간: " + status.getTimeString(), 20, 85);
}
```

(3) paintComponent의 변경점

스테이지, 이동 횟수, 플레이 시간을 모두 표시하도록 했으며, 글씨를 감싸는 흰색 둥근 사각형을 만들었다.

```
public void showRankingDialog() {
    String[] options = { "플레이 시간 랭킹", "이동 횟수 랭킹" };
    int sel = JOptionPane.showOptionDialog(this, "확진할 랭킹 종류를 선택하세요.", "랭킹 보기", JOptionPane.DEFAULT_OPTION,
        JOptionPane.INFORMATION_MESSAGE, null, options, options[0]); //버튼 두 개 팝업은 JOptionPane의 기능 - 요이머의 티스토리 참고
    if (sel == 0)
        showRankingByTime();
    else if (sel == 1)
        showRankingByMove();
}

public void showRankingByTime() {
    RankingEntry[] arr = status.getRankingsByTime();
    String msg = "플레이 시간 랭킹\n";
    for (int i = 0; i < arr.length; i++) {
        int min = arr[i].playSeconds / 60;
        int sec = arr[i].playSeconds % 60;
        msg += (i + 1) + ". " + arr[i].name + " - " + String.format("%02d:%02d", min, sec) + "\n";
    }
    JOptionPane.showMessageDialog(this, msg);
}

public void showRankingByMove() {
    RankingEntry[] arr = status.getRankingsByMove();
    String msg = "이동 횟수 랭킹\n";
    for (int i = 0; i < arr.length; i++) {
        int min = arr[i].playSeconds / 60;
        int sec = arr[i].playSeconds % 60;
        msg += (i + 1) + ". " + arr[i].name + " - " + arr[i].moveCount + "\n";
    }
    JOptionPane.showMessageDialog(this, msg);
}
```

(4) 랭킹 출력 함수

랭킹을 출력하는 함수들을 작성했다. 버튼이 두 개 있는 팝업창을 JOptionPane의 기능으로, 개발자 티스토리를 참고했다. 랭킹 계산은 GameStatus 클래스에서 하고, 랭킹 요소를 위한 RankingEntry 클래스도 있다.

2-6. GameStatus.java

```
package term_project_final;
import javax.swing.*;

public class GameStatus {

    private int playSeconds = 0;
    private Timer timer;

    private int stageNum = 0; // 0부터 시작
    private int moveCount = 0;

    private RankingEntry[] rankings = new RankingEntry[10];
    private int rankingCount = 0;

    private String lastPlayerName = "";

    public GameStatus() {
    }

    //시간을 출력하는 함수 - corderanch 사이트 참고
    public void startTimer(JPanel panel) {
        timer = new Timer(1000, e -> {
            playSeconds++;
            panel.repaint();
        });
        timer.start();
    }

    public void stopTimer() {
        timer.stop();
    }

    public void resetTimer() {
        timer.stop();
        playSeconds = 0;
        timer.start();
    }
}
```

(1) 플레이 시간, 스테이지, 이동횟수, 닉네임 등

다양한 게임 화면의 변수들을 전체적으로 관리하는 클래스를 따로 만들었다. 스테이지, 이동횟수, 닉네임 등을 이용한 함수들은 어렵지 않았지만 플레이 시간을 나타내는 것이 교재에 보이지 않았기 때문에 corderanch 사이트를 참고해 작성했다.

nextStage() 함수를 boolean 형태로 작성해 함수가 실행됐을 때 평소에는 stageNum을 증가시키고 true를 반환하다가, stageNum이 MaxstageNum과 같으면 false를 반환하도록 했다.


```

public void addRanking(String name) {
    if (rankingCount < rankings.length) {
        rankings[rankingCount++] = new RankingEntry(name, moveCount, playSeconds);
    } else {
        for (int i = 1; i < rankings.length; i++)
            rankings[i - 1] = rankings[i];
        rankings[rankings.length - 1] = new RankingEntry(name, moveCount, playSeconds);
    }
}

public RankingEntry[] getRankingsByTime() {
    RankingEntry[] copy = new RankingEntry[rankingCount];
    for (int i = 0; i < rankingCount; i++)
        copy[i] = rankings[i];
    for (int i = 0; i < copy.length - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < copy.length; j++)
            if (copy[i].playSeconds < copy[minIdx].playSeconds)
                minIdx = j;
        RankingEntry tmp = copy[i];
        copy[i] = copy[minIdx];
        copy[minIdx] = tmp;
    }
    return copy;
}

public RankingEntry[] getRankingsByMove() {
    RankingEntry[] copy = new RankingEntry[rankingCount];
    for (int i = 0; i < rankingCount; i++)
        copy[i] = rankings[i];
    for (int i = 0; i < copy.length - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < copy.length; j++)
            if (copy[i].moveCount < copy[minIdx].moveCount)
                minIdx = j;
        RankingEntry tmp = copy[i];
        copy[i] = copy[minIdx];
        copy[minIdx] = tmp;
    }
    return copy;
}

public void setLastPlayerName(String name) {
    lastPlayerName = name;
}

public String getLastPlayerName() {
    return lastPlayerName;
}

```

(2) 랭킹 계산

GameMap에서 나타내는 랭킹을 계산하는 코드를 GameStatus 클래스에 작성했다.

2-6. RankingEntry.java

```

package term_project_final;

public class RankingEntry {
    public String name;
    public int moveCount;
    public int playSeconds;

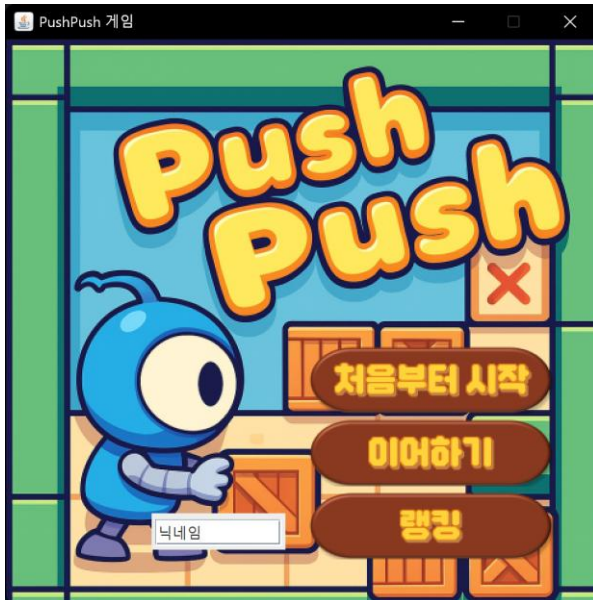
    public RankingEntry(String name, int moveCount, int playSeconds) {
        this.name = name;
        this.moveCount = moveCount;
        this.playSeconds = playSeconds;
    }
}

```

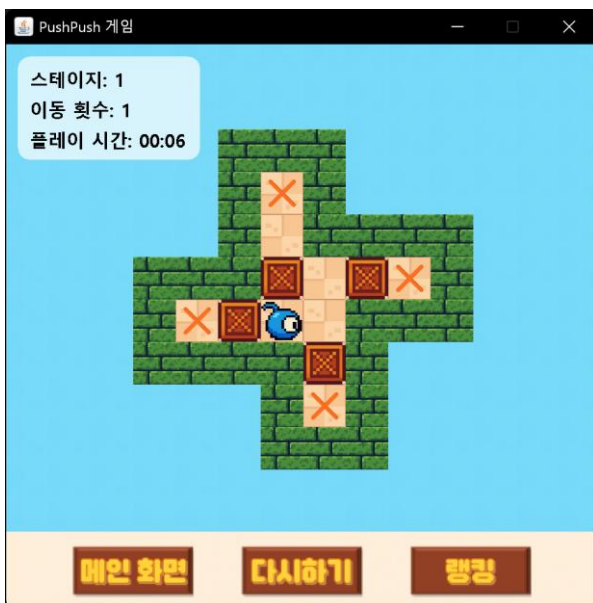
(1) 랭킹 계산을 위한 클래스

랭킹 계산을 위해 RankingEntry 클래스를 따로 만들어 GameStatus의 랭킹 계산 함수에서 랭킹 객체를 생성하도록 했다.

3. 배경화면 및 버튼, 게임 이미지, 효과음



- (1) 메인 화면의 배경화면은 AI의 이미지 생성 기능을 사용했고, 이외에 처음부터 시작, 이어하기, 랭킹 버튼은 전부 포토샵으로 작업했다.



- (2) 게임 화면의 각 블록 이미지는 AI로 이미지를 생성하고 포토샵 작업을 통해 재가공했으며, 메인 화면, 다시하기, 랭킹 버튼은 전부 포토샵으로 작업했다. 또, 버튼 아래에 깔린 이미지는 포토샵으로 간단하게 작업했다.

(3) 게임에 적용한 모든 효과음은 pixabay에서 non copyright 표시가 되어있는
게임 효과음 mp3 파일을 사용했다.

4. 중간 보고서에서의 목표 달성 여부

중간 보고서를 작성하면서 작성한 목표는 다음과 같다. 1. KeyListener 대신 KeyAdapter를 사용할 것, 2. 다시하기 버튼으로 맵을 리셋할 것, 3. 기본 기능을 모두 동작하게 하고, 객체 지향적으로 작성하기 위해 여러 개의 class 파일로 나눌 것.

먼저 첫째, 클래스를 나누면서 KeyListener가 아닌 익명 클래스를 통한 KeyAdapter를 사용할 수 있게 되었다. 둘째, 다시하기 버튼으로 맵을 리셋할 수 있게 했다. 셋째, 기본 기능이 모두 동작하고, 클래스를 각각 나눠 모든 클래스가 객체 지향적으로 동작하도록 작성하였다.

추가로 메인 화면 이미지, 버튼의 이미지 및 이미지 변경, 커서를 대거나 눌렀을 때의 효과음, 스테이지 클리어 효과음, 전체 클리어 효과음, 메인 화면 효과음, 게임 화면 효과음 등을 삽입했다. 버튼도 MouseAdapter를 익명 클래스로 작성했다.

5. 마무리하면서

자바와 객체지향에 대해 배우고 텀프로젝트를 바로 하게 됐는데, 자바와 객체지향에 대해 배운다고 해서 바로 코드를 유창하게 쓸 수 있는 것은 아니라는 것을 알게 되었다. 교재에 나오는 것 외에는 검색을 통해 배우면서 다양한 코드를 작성할 수 있었으며 심지어 다른 라이브러리를 불러와 보기도 하면서 앞으로 배워야 할 것이 무궁무진하다는 것을 깨달을 수 있었다. 방학에는 다른 프로젝트를 몇 개 해보면서 역량을 더 키워나가고 싶다.